Universitat Politècnica de Catalunya
Facultat de Matemàtiques i Estadística

Master in Statistics And Operations Research

Master's thesis

Design and Implementation of a Bluesky Simulator for Recommender Analysis

Pau Soler Valadés

Supervised by Jana Lasser and Esteve Codina Sancho

June, 2026

Abstract

English

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

Catalan

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

Spanish

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum

necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

Contents

1. Introduction 1
2. Context 2
   2.1. What is a Microblogging Social Network 2
   2.2. Why Bluesky? 3
   2.3. Problem Description 3
3. Model 4
   3.1. Narrowing Down 4
      3.1.1. Network Entities and Topology 4
      3.1.2. Relational Dynamics 5
      3.1.3. User Activity and Timeline Construction 6
      3.1.4. User State and Session Dynamics 7
      3.1.5. User Decisions and Policy 7
   3.2. Alternativa basada en funcions 9
   3.3. Network Entities and Topology 9
   3.4. Relational Dynamics and Edge Properties 9
   3.5. Network Latencies 10
   3.6. User Session Dynamics 10
   3.7. User Activity and Timeline Construction 11
      3.7.1. User Decisions and Policy 11
4. Methodology 12
   4.1. Discrete Event Simulation 12
   4.2. Technology Approach 12
   4.3. Technology Approach 13
   4.4. Random Number Generation 13
      4.4.1. Ziggurat Algorithm 13
      4.4.2. Categorical Distribution 14
   4.5. Dynamic Graphs 15
5. Design 16
   5.1. Assumptions 16
   5.2. Mechanics 16
      5.2.1. User Decision Policy 17
      5.2.2. Session and Timelines 17
      5.2.3. Delays and Quicks 17
   5.3. Data Structures 18
   5.4. Pseudocode 18
   5.5. Implementation 20
      5.5.1. Data Oriented Design Basics 20
      5.5.2. Dynamic Dispatch of Distributions via JSON 21
      5.5.3. Heap and n-ary Leaf 21
      5.5.4. Post Creation and Storage 21
      5.5.5. Graph Representation of the Network Topology 22

5.5.6. Memory Allocation Strategies 22
5.6. Additional Mechanics: Quoting, Notifications and Replies 22
5.7. Trace Validation 23
6. Content approach: Embeddings 24
6.1. Main Mechanics and match with DES 24
6.2. Sampling Mechanisms 25
6.3. Action Independence 25

1. Introduction

We'll do this at the end :)

Things that must appear for sure:

Context: what other papers have attempted while doing discrete event simulations and social networks (not that many, as far as I know) and **justification**: we really need to figure out things about social networks, cause they influence out daily lives a lot now.

Objectives

Definir i i implementar un model que ressembli les principals característiques d'una xarxa social de microblogging. L'objectiu és poder establir quines característiques fan que un post sigui més "popular" que un altre. Definiició popular: viu més estona…

1. Create a Discrete event simulation, and verificate its correctness with some strategies with strong assumptions (let's call this for now the engine, which should be no machine learning, user homogeneity and just the main mechanincs)
2. Add drop the content assumption with embeddings and all the stuff.
3. Results of the simulaton
4. Way to compare them with real life, to see if it makes sense (Macro metrics as lifetime, distance of post travel)

Hypothesis A discrete event simulation is a good model for a real social network (probably not) There are some post characteristics of the content of the posts that they make them much more likely to be shared.

Research Question

- Is a DES a good way to model a microblogging social network?
- How can we introduce a meaningful way to detect similarities of posts and users?
- Which are appropiate mechanism to make user behaviour (when seeing a post) change according to the contents of the post?
- Are some post contents infuencial to how far is the post going to travel?

Report structure How many chapters and all this stuff.

OBJECTIVE: study the propagation of posts according to its contents and determine if there are characteristics that makes some posts more popular than others.

2. Context

This section introduces context to the project: what a microblogging social network is, the nature of the phenomenon we want to study, and why Bluesky is the chosen social network to simulate.

2.1. What is a Microblogging Social Network

Despite social networks being a relatively new addition to normal modern life, they have fundamentally changed how information is consumed and spread in the modern age. To adequately understand the aims of this project, some definitions and context regarding social networks are provided.

Definition: A **Social Network** is a structure made up of a set of actors (such as individuals or organizations) and the dyadic ties (interactions or relationships) between them. In a digital context, it is a platform that enables users to communicate, share information, and form communities.

This project tackles a very specific kind of social network: the microblogging network.

Definition: A **Microblogging Social Network** (e.g., Twitter/𝕏 or Bluesky) is a specialized type of social network where users publish and exchange short-form content. This is enforced by a limit on the maximum number of characters per entry, known as microblogs or posts. A post, while traditionally text-based, can also include up to four multimedia elements.

[TODO: buscar sources per aquestes dues definitions]

To model a microblogging social network, it is imperative to understand and describe all the features that compose the application. Specifically, the following description is of the microblogging platform Bluesky (TODO: Cite bluesky), as it is the platform chosen to simulate (see Section 2.2).

The post is the fundamental building block of Bluesky, acting as the primary vehicle for the information that will be spread. A user sees these posts in a feed, which is a sorted collection of posts categorized by specific rules. For the feed to contain posts, we must explain the other fundamental relationship in a social network: the follow.

[TODO: Picture of the feed of Bluesky]

A user can follow other users, allowing the content of those users to appear to the "follower". That is, if user u follows user v, all the posts that user v creates will appear in the feed of user u. This is one mechanism for a user's feed to populate with posts. The other way for content to travel is how a user interacts with a post once it appears in their feed, this being the main source of post appearance in the user feed. Conversely, there are some actions a user can perform over other users which limit the posts that can appear on a user's feed: a user can block (preventing both users from seeing each other's activity) or mute (activity of a muted user is not shown to the user) another user, which will alter the posts that can appear on the timeline of the user who performed that action.

A user can meaningfully act on a post in four different ways:

- **Repost:** If user u reposts a post in their feed, that post will subsequently appear in the feeds of user u's followers.

- **Like:** If a user likes a post, it won't show in their followers' feeds, but it will be stored in the user's profile (discussed later).
- **Quote:** This is a direct answer attached to a post. It is shown together with the original post in the followers' timeline, acting as commentary on the original content.
- **Reply:** This is a direct response to an original post. The reply will be shown in the followers' feed and clearly marked as a reply, though the original post usually won't be shown alongside it unless the follower also follows the original author.

[TODO: Show pictures of how a quote is shown, how a reply is shown]

Lastly, every user has a profile, which is customizable with a profile picture, a description, and a background image. The profile acts as a public ledger containing all posts the user has written or reposted, all replies made to other posts, and all likes given.

Bluesky, as an open platform, allows for user-made feeds with diverse rules and categories dictating which posts are shown. These include feeds focused on highly specific topics (such as technology, local events, or art). The two primary feeds provided by default are the *Discover* feed and the *Following* feed.

The Discover feed uses a recommendation algorithm to suggest the most relevant posts to a user based on their tastes and the network of people they follow. This criterion usually excludes the temporal component of when posts were created or reposted, focusing instead on content similarity and engagement.

The Following feed is a traditional social network timeline. It displays creations and reposts exclusively from the accounts the user follows, showing them in strict reverse-chronological order, from newest to oldest.

[TODO: Picture of the tabs showing the different feeds the Bluesky app has]== Model

2.2. Why Bluesky?

TBD: Bluesky is open, talk about ATProto, and talk about how the firehose works to obtain the data

Bluesky is …

2.3. Problem Description

The problems with social networks are bla bla and I did all of this to try to solve them.

3. Model

This section narrows the description of the Bluesky social network (see Section 2.1) into the most important mechanics to study the objective and defines it as a Time-Evolving Heterogeneous graph.

3.1. Narrowing Down

To model all the features in a complete social network is a challenge out of the scope of this project. To achieve the objectives of the project within its time constraints, a significant subset of the Bluesky platform will be modeled according to the research tools and approaches to be followed.

1. Just the Following Feed

The following feed is a timeline with a reverse-chronological post showing criteria, and from now on this will be referred to as the *timeline* of every user. As simulating a recommender is a difficult challenge in itself, it is believed that the flow of information can be meaningfully studied with a more traditional content strategy, even if the use of these feeds is not the norm on social networks.

2. Static Users and Followers

During the course of the simulation, no new users will be added, nor new relationships between them. The inter-user relationships are considered static during the whole duration of the simulation, as the flow of content can be studied without this behavior.

3. No Mutes nor Blocks

We assume that if user u follows user v, user u will receive all posts from user v.

4. No Quotes, no Replies

With the same criteria, we want to focus on the content of the text, so the way the information transmits itself can be reduced to a single post.

5. No Profile of a User

A user won't be able to enter to see other users' profiles; they will be limited to observing their posts on the timeline.

Let's define which features of Bluesky are going to be modeled in the simulation:

1. Users can act over a post by liking or reposting it.
2. Users see posts in a timeline: posts will be seen in reverse-chronological order from the accounts they follow.

To model these dynamics, this section introduces a unified mathematical notation that models the microblogging platform as a Time-Evolving Heterogeneous Graph [TODO: MISSING QUOTATION] and the reasons for that are given in the section [methodology/graph]. This formulation rests on the acknowledgment that there are two distinct entities, users and posts, as well as different types of edges to characterize the relationships between entities of the same type and different types. The relationships between the entities are, by their very nature, changing over time.

3.1.1. Network Entities and Topology

The model consists of two entities: users $\mathcal{U}$ and posts $\mathcal{I}$[1].

Definition (Nodes): The nodes of the graph are the union of all the participating entities $V = \mathcal{U} \cup \mathcal{I}$.

Due to assumption number two, we can also state that $|\mathcal{U}| = N$, as the number of users will remain stable during the duration of the simulation. As followers also remain stable during the whole run of the simulation, we can define followers and followees.

In contrast to the number of users, the number of posts is variable as users can create posts at specific timestamps. There is no need for a formal definition of time and we'll just consider $T = \mathbb{R}^+$.

Definition (User posts): We denote the set of all posts created by user u up to time $t \in T$ as $\mathcal{P}(u, t)$

Of course, now we can express all the posts in the network up to the time t as

$$\mathcal{I}(t) = \bigcup_{u \in \mathcal{U}} \mathcal{P}(u, t)$$

Què tal esteve! Aprofito per comentar-te que et deixo coses en color vermell com a preguntes. Que el conjunt de posts tingui una t com a funció se'm fa molt lleig, però és així com funciona això. Hi ha alguna manera més maca? El que he fet en algun moment és $\mathcal{P}_t(u)$, és a dir, posar el temps com a subindex.

3.1.2. Relational Dynamics

Every event occurring during the simulation is depicted as an edge within the graph. The nature of these edges varies significantly depending on the action performed and the specific entities involved. There are two types of relationships:

Definition (User-to-User): we denote $\mathcal{R}_{\mathcal{UU}}$ as the set with all the possible relationships between users. Specifically, $\mathcal{R}_{\mathcal{UU}} = \{\text{follow}\}$.

Definition (User-to-Post): we denote $\mathcal{R}_{\mathcal{UI}}$ as the set with all the possible actions a user can perform over a post $\mathcal{R}_{\mathcal{UI}} = \{\text{create}, \text{like}, \text{repost}, \text{ignore}\}$

The edges of the graph then are all the union of edges of the two relationships.

Definition (Edges): we denote the set of edges $E = E_{\mathcal{UU}} \cup E_{\mathcal{UI}}$, where

$$E_{\mathcal{UI}} = \{(u, i, r, t) \mid u \in \mathcal{U}, i \in \mathcal{I}, r \in \mathcal{R}_{\mathcal{UI}}, t \in T\}$$

$$E_{\mathcal{UU}} = \{(u, v, \text{follow}, 0) \mid u \in \mathcal{U}, v \in \mathcal{U}\}$$

Due to the nature of the create relationship and the definition of $\mathcal{P}$, given that $t^- < t$, the following is true:

$$(u, i, \text{create}, t) \in E \Rightarrow i \notin \mathcal{I}(t^-) \wedge \mathcal{I}_t = \mathcal{I}(t^-) \cup \{i\}$$

Now, given the definition of Time-Evolving Heterogeneous Graph

Definition (Time-Evolving Heterogeneous Graph): A graph $G = (V, E)$ where the node set V comprises multiple distinct entity types, $V = \mathcal{U} \cup \mathcal{I}$, and the edge set E consists of heterogeneous relationships occurring over a temporal dimension T, such that each edge is represented as a tuple $e = (v_{\text{src}}, v_{\text{dst}}, r, t)$ with $t \in T$.

It is clear that $G = (V(t), E)$ complies with that definition.

[1] the nomenclature $\mathcal{I}$ stems from recommender theory, and stands for items

From now on, time t will be included in the definitions, as for a specific e to belong or not in E depends on the specific time.

Lastly, we can define the users that a specific user u is following, and the users that follow them:

Definition (Followees / Following): The subset of users that user $u \in \mathcal{U}$ is following is denoted as

$$\mathcal{N}_{\text{out}}(u) = \{v \in \mathcal{U} \mid (u, v, \text{follow}, 0) \in E\}.$$

This dictates the sources of information populating user u's timeline, and coincides with the concept of the out-neighborhood of a node in graph theory.

[TODO: Exemple de followers i followee]

Definition (Followers): The subset of users that follow user u is denoted as

$$\mathcal{N}_{\text{in}}(u) = \{v \in \mathcal{U} \mid (v, u, \text{follow}, 0) \in E\}.$$

These are the users affected by user u's actions.

3.1.3. User Activity and Timeline Construction

A critical aspect of the simulation is accurately modeling what a user perceives. To build a user's timeline, we first define the cumulative activity of any user u up to a specific time t as the collection of posts they have actively propagated.

Definition (User Activity): We define the active footprint of a user $\mathcal{A}(u, t)$ as all posts for which the user has performed a spreading action (repost or create) prior to time t.

$$\mathcal{A}(u, t) = \{i \in \mathcal{I} \mid \exists e = (u, i, r, \tau) \in E \text{ where } \tau < t \text{ and } r \in \{\text{repost}, \text{create}\}\}$$

All the posts a user can interact with are contained in their timeline. A naive approach would be to simply aggregate the active footprints of everyone a user follows. However, this creates an unnatural artifact: if user u_1 creates a post, and their followee u_2 reposts it, that post would loop back into u_1's feed.

To reflect realistic platform behavior, a user's timeline must not contain their own authored content. However, users *can* re-encounter external posts they have previously seen or interacted with.

Definition (Timeline): We define the timeline for user u at time t, denoted as $\mathcal{T}(u, t)$, as the union of all activities from their followees, strictly excluding the user's own original creations $\mathcal{P}(u, t)$:

$$\mathcal{T}(u, t) = \left(\bigcup_{v \in \mathcal{N}_{\text{out}}(u)} \mathcal{A}(v, t) \right) - \mathcal{P}(u, t)$$

This formulation enforces a no-self-loop condition for authored content while permitting natural, repeated exposures to viral posts across the network.

To systematically process this timeline, standard retrieval functions are employed over any time-sorted set S:

- $\text{newest}(S)$: retrieves the item associated with the smallest timestamp.
- $\text{pop}(S)$: retrieves the item associated with the largest timestamp.
- $S(t)$: denotes the subset of elements with timestamps strictly preceding t.

Addendum: Temporal Dynamics

While the structural equations above dictate *what* belongs in a timeline, the simulation also accounts for temporal realism. Information dissemination is not instantaneous. The model incorporates a propagation delay (the time required for a followee's action to surface on an observer's timeline) and an interaction delay (the cognitive processing time before a user reacts). These delays are applied as temporal offsets to the timestamps τ during the simulation's event-handling phase, ensuring interactions follow a realistic chronological flow.

3.1.4. User State and Session Dynamics

Having established the timeline construction, we must define the temporal constraints governing a user's ability to interact with it. While the flow of information in the network is continuous, individual user engagement occurs in discrete, contiguous sessions.

Definition (User State): We define a time-dependent state function $S(u, t)$ that maps a user at a specific time to their current operational status:

$$S : \mathcal{U} \times T \to \{\text{online}, \text{offline}\}$$

This state strictly dictates the permissible network dynamics for user u at any given moment:

1. **Offline State:** When $S(u, t) = \text{offline}$, the user is disconnected and cannot interact with the network. Formally, no action edges originating from u can be generated at time t:
$$S(u, t) = \text{offline} \Rightarrow \nexists i \in \mathcal{I}, r \in \mathcal{R}_{\mathcal{U}\mathcal{I}} \text{ such that } (u, i, r, t) \in E$$
2. **Online State:** When $S(u, t) = \text{online}$, an active session begins. The user is now capable of processing their accumulated timeline and executing actions (create, repost, like, or ignore). Consequently, the existence of any interaction edge inherently requires the user to be online:
$$(u, i, r, t) \in E \Rightarrow S(u, t) = \text{online}$$

The online/offline state forces a dynamic where online and offline periods are intercalated. The user's timeline $\mathcal{T}(u, t)$ when $S(u, t) = \text{offline}$ accumulates posts generated by $\mathcal{N}_{\text{out}}$, effectively acting as an unread queue. When $S(u, t^+) = \text{online}$, if there are enough users online, $\mathcal{T}(u, t) \neq \varnothing$, so they will have content to look at and explore. This emulates the behavior of social networks quite well, when the user gets fatigued, disconnects, and rechecks the timeline later.

3.1.5. User Decisions and Policy

The interactions within the network are driven by the decision-making processes of the users. We model user behavior through a policy π, which governs the probability of executing specific actions. Because consuming existing content and generating new content are fundamentally different mechanisms, the policy is bifurcated into two distinct components: a reactive policy and a spontaneous generative policy.

1. **Reactive Policy (User-to-Post):** When an online user is presented with a post i in their timeline $\mathcal{T}(u, t)$, they must decide how to interact with it. We define the reactive policy π_{act} as a probability distribution over the subset of timeline-permissible actions, $\mathcal{R}'_{\mathcal{U}\mathcal{I}} = \{\text{ignore}, \text{like}, \text{repost}\}$.
$$\sum_{a \in \mathcal{R}'_{\mathcal{U}\mathcal{I}}} \pi(a) = 1$$
2. **Spontaneous Policy (Creation):** The act of creating a new post does not depend on the contents of the timeline; it is a spontaneous event generated by the user. We define the

generative policy λ as the probability (or rate) at which an online user decides to execute the create action during a given time step or session, introducing a completely new item into $\mathcal{I}_t$.

3.2. Alternativa basada en funcions

Hola Esteve. Resulta que he trobat un article que defineix exactament el tipus de graph que estem fent servir, molt més general, i és bé, diferent però de la mateixa manera molt elegant. M'he molestat (amb un copet d'ajuda de la IA) a reescriure-ho tot però amb aquesta formulació alternativa. Jo crec que faria aquesta segona, però no ho tinc clar.

The model of the simplification of bluesky is a Time-Varying Heterogeneous Graph:

Definition (Time-Varying Heterogeneous Graph): Having established the temporal properties of our entities and their relationships, we formally define our system as a Time-Varying Graph $\mathcal{G} = (V, E, T, \rho, \psi, \eta)$. Here, V and E form the universal topological space, ψ and ρ govern the temporal existence of nodes and edges respectively, and η bounds the chronological flow of information across the network.

The following text defines and maps all the functions and sets according to this given definition.

3.3. Network Entities and Topology

The model consists of two entities: users $\mathcal{U}$ and posts $\mathcal{I}$[2].

Unlike traditional dynamic graphs where the set of vertices grows, we define the graph over the universe of all entities that will ever participate in the simulation.

Definition (Universal Nodes): The node set V is the static union of all participating entities throughout the entire simulation lifecycle: $V = \mathcal{U} \cup \mathcal{I}$.

To represent the temporal reality of posts being created, we introduce a node presence function ψ. Given the continuous time domain $T = \mathbb{R}^+$, the presence function dictates whether a node exists at time t:

$$\psi : V \times T \to \{0, 1\}$$

Due to our simulation assumptions, the user base remains stable, meaning $\forall u \in \mathcal{U}, \forall t \in T, \psi(u, t) = 1$. In contrast, a post $i \in \mathcal{I}$ is intrinsically tied to its creation timestamp t_c. Thus, its presence is a step function:

$$\psi(i, t) = \begin{cases} 1 \text{ if } t \geq t_c \\ 0 \text{ otherwise} \end{cases}$$

We can now cleanly define the set of available items at any time t simply as $\mathcal{I}_t = \{i \in \mathcal{I} \mid \psi(i, t) = 1\}$.

l'existència de ρ elimina la necessitat de tenir un conjunt en funció del temps $\mathcal{I}(t)$

3.4. Relational Dynamics and Edge Properties

Similar to the node set, we define a universal edge set E containing every potential interaction between entities. There are two types of relationships: $\mathcal{R}_{\mathcal{U}\mathcal{U}} = \{\text{follow}\}$ and $\mathcal{R}_{\mathcal{U}\mathcal{I}} = \{\text{create}, \text{like}, \text{repost}, \text{ignore}\}$.

Definition (Universal Edges): We denote the set of all possible edges $E = E_{\mathcal{U}\mathcal{U}} \cup E_{\mathcal{U}\mathcal{I}}$, where

[2] the nomenclature $\mathcal{I}$ stems from recommender theory, and stands for items

$$E_{\mathcal{UI}} = \{(u, i, r) \mid u \in \mathcal{U}, i \in \mathcal{I}, r \in \mathcal{R}_{\mathcal{UI}}\}$$
$$E_{\mathcal{UU}} = \{(u, v, \text{follow}) \mid u \in \mathcal{U}, v \in \mathcal{U}\}$$

To capture the specific temporal dynamics of these connections, we define two continuous-time functions over the edge set: the edge presence function ρ and the latency function η.

Definition (Edge Presence): The function $\rho : E \times T \to \{0, 1\}$ indicates if an interaction or connection is active at a given time. Its behavior depends on the edge type:

1. **Structural Edges** ($E_{\mathcal{UU}}$): A follow relationship initiated at t_f persists, meaning $\rho((u, v, \text{follow}), t) = 1$ for all $t \geq t_f$.
2. **Event Edges** ($E_{\mathcal{UI}}$): Actions upon items are punctual events. If user u performs action r on item i exactly at time t_e, then $\rho((u, i, r), t_e) = 1$, and 0 otherwise. Specifically, the create relationship triggers the node presence of a post: $\rho((u, i, \text{create}), t_c) = 1 \Rightarrow \psi(i, t) = 1$ for all $t \geq t_c$.

3.5. Network Latencies

Comparat amb la formulació anterior, ara els delays ja no són una invenció, sinó part de la formulació directa, cosa molt més elegant.

Definition (Edge Latency): Information dissemination and user reactions are not instantaneous. We define a latency function $\eta : E \times T \to T$ that maps every edge to a specific temporal delay based on its interaction type:

- **Propagation Delay**: Associated with structural follow edges $e \in E_{\mathcal{UU}}$, dictating the time required for a followee's action to surface on the observer's timeline.
- **Interaction Delay**: Associated with reactive event edges $e = (u, i, r)$ where $r \in \{\text{like}, \text{repost}, \text{ignore}\}$, representing the cognitive processing time before a user reacts to a post.
- **Creation Delay**: Associated with generative event edges $e = (u, i, \text{create})$, representing the time taken to compose and publish a new item.

3.6. User Session Dynamics

Individual user engagement occurs in discrete, contiguous sessions. Rather than defining an external state function, these sessions act as an intrinsic structural constraint on the edge presence function ρ.

Definition (User Sessions): We define the periods a user u is online as a subset of time $\mathcal{O}(u) \subset T$. The edge presence function for any reactive or generative event is strictly constrained by this subset. If a user is offline, no action edges can be generated:

$$t \notin \mathcal{O}(u) \Rightarrow \forall i \in \mathcal{I}, \forall r \in \mathcal{R}_{\mathcal{UI}}, \rho((u, i, r), t) = 0$$

Consequently, the existence of any event edge inherently requires the user to be in an active session:

$$\rho((u, i, r), t) = 1 \Rightarrow t \in \mathcal{O}(u)$$

Lastly, we can define the users that a specific user u is following, and the users that follow them:

Definition (Following): The subset of users that user $u \in \mathcal{U}$ is following, assuming connections are established at $t = 0$, is denoted as

$$\mathcal{N}_{\text{out}}(u) = \{v \in \mathcal{U} \mid \rho((u, v, \text{follow}), 0) = 1\}.$$

This dictates the sources of information populating user u's timeline, and coincides with the concept of the out-neighborhood of a node in graph theory.

[TODO: Exemple de followers i followee]

Definition (Followers): The subset of users that follow user u, assuming connections are established at $t = 0$, is denoted as

$$\mathcal{N}_{\text{in}}(u) = \{v \in \mathcal{U} \mid \rho((v, u, \text{follow}), 0) = 1\}.$$

These are the users affected by user u's actions.

3.7. User Activity and Timeline Construction

To construct a user's timeline, we extract the historical footprint of the network using the edge presence function ρ, while accounting for the delays defined by η.

Definition (User Activity): The active footprint of a user $\mathcal{A}_{t(u)}$ includes all items the user has actively propagated prior to time t.

$$\mathcal{A}_{t(u)} = \{i \in \mathcal{I} \mid \exists e = (u, i, r) \in E \text{ where } \rho(e, \tau) = 1 \text{ for some } \tau < t \text{ and } r \in \{\text{repost}, \text{create}\}\}$$

Definition (Timeline): The timeline $\mathcal{T}_t(u)$ is the aggregated activity of the user's out-neighborhood $\mathcal{N}_{\text{out}}(u)$, strictly excluding items the user organically authored themselves, $\mathcal{P}_{t(u)}$. The time at which an item from followee v appears in u's timeline is offset by the propagation delay $\eta((u, v, \text{follow}), t)$:

$$\mathcal{T}_t(u) = \left(\bigcup_{v \in \mathcal{N}_{\text{out}}(u)} \mathcal{A}_{t-\eta((u,v,\text{ follow}),t)}(v) \right) - \mathcal{P}_t(u)$$

3.7.1. User Decisions and Policy

The interactions within the network are driven by the decision-making processes of the users. We model user behavior through a policy π, which governs the probability of executing specific actions. Because consuming existing content and generating new content are fundamentally different mechanisms, the policy is bifurcated into two distinct components: a reactive policy and a spontaneous generative policy.

1. **Reactive Policy (User-to-Post):** When an online user is presented with a post i in their timeline $\mathcal{T}_t(u)$, they must decide how to interact with it. We define the reactive policy π_{act} as a probability distribution over the subset of timeline-permissible actions, $\mathcal{R}'_{\mathcal{U}\mathcal{I}} = \{\text{ignore}, \text{like}, \text{repost}\}$.

$$\sum_{a \in \mathcal{R}'_{\mathcal{U}\mathcal{I}}} \pi(a) = 1$$

2. **Spontaneous Policy (Creation):** The act of creating a new post does not depend on the contents of the timeline; it is a spontaneous event generated by the user. We define the generative policy λ as the probability (or rate) at which an online user decides to execute the create action during a given time step or session, introducing a completely new item into $\mathcal{I}_t$.

En definitiva, aquesta segona sembla bastant millor tu

4. Methodology

This section covers …

4.1. Discrete Event Simulation

Esteve, si tens alguna font per citar això que sigui més pro digues-me-la

Discrete Event Simulation (DES) is a modeling methodology that represents the operation of a complex system as a chronological sequence of events. In a DES model, the system's state only changes at discrete points in time when a specific event occurs, allowing the simulation engine to jump efficiently from one event to the next without calculating the time in between.

It is useful to contrast this methodology with recent developments in Agent-Based Modeling (ABM). Modern ABM is increasingly utilized to explore complex, highly interactive environments, such as social network dynamics simulated via Large Language Models (LLMs) [1]. Furthermore, while some ambitious initiatives—such as the DESIRE project at the University of Graz—are exploring ABM as a pathway to develop accurate "digital twins" of real-world systems, the methodology fundamentally revolves around generating bottom-up, emergent macro-behavior from the micro-level interactions of stateful agents.

DES, conversely, takes a top-down, process-centric approach. Instead of explicitly simulating the microscopic interactions of individual agents, DES abstracts entities into continuous flows governed by probability distributions and queueing theory [2]. Because it relies fundamentally on statistical abstraction rather than structural agent simulation, DES guarantees the validity of its results through two strict statistical assumptions: the simulation must process enough volume (entities) for the outcomes to be extrapolable, and the model must be replicated a sufficient number of times for those probabilistic distributions to achieve statistical convergence.

4.2. Technology Approach

Because the validity of a DES relies entirely on sheer computational volume and repeated runs to achieve statistical significance, a suboptimal implementation can easily contradict the underlying assumptions needed to guarantee a successful process. Execution speed, deterministic behavior, and tightly optimized computational loops are paramount.

The first step in implementation is choosing the appropriate tools for the job. Interpreted languages like Python and R are immediately discarded; even on powerful hardware, the overhead of interpretation introduces unacceptable latency for massive, CPU-bound simulation loops. Following this logic, manual memory management becomes necessary to deeply optimize performance and take full advantage of CPU caching. This requirement effectively rules out garbage-collected languages such as Java or Go, which cannot guarantee the deterministic, low-latency execution and exact memory layout control required here. With requirements pointing strictly toward a systems language with manual memory management, the choice of Zig was clear.

Zig is a general-purpose programming language and toolchain designed for maintaining robust, optimal, and reusable software. By design, it provides C-like performance alongside modern quality-of-life improvements and strict guardrails against common C pitfalls, such as

segmentation faults and null pointer dereferences. With the application of the right memory optimization techniques (see [TODO: @ sec-des-hpc]), Zig enables highly scalable implementations that extract the maximum possible performance from the hardware.

4.3. Technology Approach

Discrete event simulations —in contrast with Agent Based Modelling— guarantee their results with two underlying assumptions: the simulation can run with enough entities to have an extrapolable result and the simulation will need to be ran a sufficient amount of times for the results to be statistically significant. This facts highight the need for having to be careful regarding implementation details, as a very suboptimal implementation might contradict both underliying assumptions to guarantee a successful process.

The first step then is choosing the appropiate tools for the job. As we need scalability and performance, interpeted lanugages as R and Python are immedialtey discarded. Even with powerful machines, the implementations will not be nor performant nor scalable. Following the same train of tought, manual memory management is required to optimize performance and take advantage of caching, which discards garbage collected languages as Java or Go. Then, the requirements aim for the use of a language with manual memory management, so the election for Zig was clear.

Zig [TODO: citar zig] is a is a general-purpose programming language and toolchain for maintaining robust, optimal and reusable software. Provides C like performance with modern quality-of-life improvements as well as much more guard ralining against common and painful C errors, such as segmentation faults and null pointer derreferences by design. Zig provides, given enought knowledge about optimization techinques, scalable implementations and the maximum performance that the computer can provide.

4.4. Random Number Generation

This section covers the implementations of the Random Number Generators needed in the main simulation, as Zig did not have a library of distributions. The distributions library has been published under the MIT license and its source available [TODO: Citar el repository de distirbuitons]

4.4.1. Ziggurat Algorithm

The generation of random variates for continuous distributions, specifically the Normal and Exponential distributions, relies on the highly optimized Ziggurat algorithm [TODO: Cite ziggurat algorhtim]. This method is a form of rejection sampling that overlays the target probability density function (PDF) with a set of $n = 256$ horizontal rectangles (called "ziggurat" to the resemblance to the building with the same name TODO: Cite ziggurat) of equal area, constructed such that they tightly bound the distribution curve.

Our implementation in Zig heavily leverages compile-time evaluation (`comptime`) to specialize the algorithm identically for both `f32` and `f64` precision without runtime overhead. The core optimization focuses on minimizing calls to the pseudo-random number generator (PRNG).

Instead of requiring two distinct random values—one to select a rectangle and another to sample a point within it—a single 64-bit random integer is generated (or 32-bit for `f32`).

From this single random word, two values are extracted with zero PRNG overhead:

1. The lowest 8 bits are masked (`bits & 0xff`) to uniformly select the index i of one of the 256 precomputed rectangles.
2. The remaining 52 bits are shifted and directly utilized as the mantissa of an IEEE 754 floating-point number. [TODO: cite the IEEE soruce]

To construct the uniform floating-point value efficiently, the integer mantissa is bitwise OR-ed with a predefined exponent mask. For symmetric distributions like the Normal, the exponent is chosen such that the resulting float falls into the interval $[2, 3)$. Subtracting 3 then shifts the domain to $[-1, 1)$. For asymmetric distributions like the Exponential, the exponent mask places the float in $[1, 2)$, and subtracting an offset near 1 yields a uniform variate in $[0, 1)$.

This uniformly distributed value u is scaled by the x-coordinate boundary of the selected rectangle i, producing a candidate sample $x = uc \cdot x_i$. If the candidate falls strictly within the core of the rectangle ($|x| < x_{\{i+1\}}$), it is immediately accepted. This fast-path covers approximately 99% of all generation requests and bypasses costly mathematical operations.

When a candidate falls outside the fast-path core, two edge cases are handled:

- **Boundary Cases:** If $i > 0$ and the sample is in the wedge between rectangles, an additional random draw evaluates the exact PDF to deterministically accept or reject the candidate.
- **Tail Cases:** If $i = 0$, the sample lies in the infinite tail of the distribution. A specialized `zeroCase` function handles this tail recursively. For the Exponential distribution, it evaluates the inverse transform shifted by the rightmost boundary R, yielding $R - ln(U)$. For the Normal distribution, it implements Marsaglia's tail generation, looping to draw values until $-2y < x^2$ is satisfied, and appropriately shifting the result by R.

4.4.2. Categorical Distribution

The categorical distribution models discrete random variables that can take on one of k possible categories, each with a specific probability. In our Zig implementation, a categorical distribution is initialized with an array of distinct items (`data`) and their corresponding probabilities (`weights`). During initialization, an accumulator array (`acc`) is computed that stores the cumulative sum of the given probabilities.

To sample from this distribution, we employ a standard inverse transform method: a uniform floating-point value $u \in [0, 1)$ is drawn and compared linearly against the cumulative weights array until a value satisfying $u \leq \text{acc}[i]$ is found, at which point the category at index i is returned.

While theoretically faster alternatives like the Alias Method [TODO: Cite alias method] exist ——capable of sampling in $O(1)$ time after an $O(k)$ setup—— they introduce additional memory overhead and initialization complexity. For the context of this simulation, where k is typically very small (e.g., modeling a handful of user action types), the performance difference is strictly negligible. Thus, we have opted for the linear search approach due to its simplicity and cache locality.

However, to optimize the performance of the linear search, a critical convention is enforced when constructing the distributions: the categories must always be sorted by their probability

in descending order. By placing the most probable outcomes at the beginning of the arrays, the cumulative sum grows rapidly, maximizing the chance that the linear search terminates in the very first iterations, thereby achieving near $O(1)$ empirical performance.

4.5. Dynamic Graphs

Esteve, no acabo d'entendre exactament què haig d'escriure aquí. Osigui perquè he necessitat un time heterogeneous graph per modelizar el problema?

Revise the articles from the beginning. This is not about the data (users and relationships) and data, but the model of the data.

5. Design

This section covers the design of the simulation program and implementation.

5.1. Assumptions

To ensure tractability and isolate specific network dynamics in the first version of this simulation, we constrain the theoretical model using the following simplifying assumptions:

1. **User Homogeneity:** Every user $u \in \mathcal{U}$ is indistinguishable in behavior and shares the exact same decision policy π and creation rate λ.
$$\forall u, v \in \mathcal{U} : \pi^u = \pi^v = \pi \quad \text{and} \quad \lambda^u = \lambda^v = \lambda$$

Note: The implementation code parses User Decision Policy and `max_posts` per-user from JSON, breaking strict theoretical user homogeneity.

2. **Post Homogeneity:** All posts are treated as content-agnostic commodities. A user's probability of executing an action a is completely independent of the specific item being evaluated:
$$\pi(a \mid i) = \pi(a \mid j) = \pi(a) \quad \forall i, j \in \mathcal{I}, \forall a \in \mathcal{R}'_{\mathcal{U}\mathcal{I}}$$
3. **Action Independence (Markovian Behavior):** A user's choice to interact with a post i at time t depends strictly on the static policy π and is independent of their historical impression history $\mathcal{H}_u^{\text{act}}$. Letting $\mathbb{P}$ denote probability:
$$\mathbb{P}\Big((u, i, a, t) \in E \;\Big|\; \mathcal{H}_{u(t)}^{\text{act}}\Big) = \pi(a)$$
4. **Timeline Consumption (Reverse-Chronological):** The timeline $\mathcal{T}(u, t)$ is consumed strictly in reverse-chronological order. We formalize the timeline as a priority structure (akin to a max-heap), where the next item presented to the user is extracted via a pop operation that retrieves the item i associated with the most recent timestamp τ_i:
$$\text{pop}(\mathcal{T}(u, t)) = \arg\max_{i \in \mathcal{T}(u,t)} \tau_i$$
5. **Session-Based Activity:** Interactions are strictly gated by the user state function $S(u, t)$. When online, actions evaluated by π and λ are executed; when offline, the user acts only as a passive accumulator of incoming timeline events.
6. **No-Self-Loop:** A user will never be served their own created content in their timeline.
$$\mathcal{P}(u, t) \cap \mathcal{T}(u, t) = \varnothing$$
7. **Single-Action Limit:** A user can meaningfully interact with a specific item i at most once.
$$(u, i, a_1, t_1) \in E \quad \text{and} \quad (u, i, a_2, t_2) \in E \Rightarrow a_1 = a_2 \quad \text{and} \quad t_1 = t_2$$

Note: The implementation actively enforces a single-impression limit via the `user_seen_post` bitset matrix, meaning users strictly cannot even view the same post twice on their timeline, implicitly enforcing the single-action limit.

5.2. Mechanics

This section stablishes which main components are in play.

5.2.1. User Decision Policy

The user decision policy pi determines which action a user takes when they see a post.

- It is implemented using a `Categorical` discrete distribution.
- During initialization, the simulation parses the JSON configuration to map a list of possible actions (`"ignore"`, `"like"`, `"repost"`) to their respective probability weights.
- When an `action` event is triggered, the simulation simply evaluates `simconf.user_policy.sample(rng)` in $O(1)$ empirical time to determine the exact action to schedule next.

5.2.2. Session and Timelines

The introduction of sessions introduces several operational challenges to the discrete event simulation, specifically regarding the interruption of stochastic processes. Time between actions and creations are measured **within a session** rather than globally.

Because DES relies on uninterrupted renewal processes to sample probability distributions correctly, pausing a user's activity when they go offline could mathematically invalidate the process (e.g., failing a goodness-of-fit test) if not handled properly. To circumvent this, the simulation associates a `session_gen` identifier to each user. Whenever a user goes offline, their timeline is cleared (as it would be refreshed upon new login) and their `session_gen` counter increments.

When the simulation pops an event from the central queue, it first checks if `event.session_gen != user.session_gen`. If this condition holds true, the event is considered **stale** (originating from a previous session that was interrupted) and is safely discarded. Note: The Zig implementation codebase explicitly calls this variable `session_id` rather than `session_gen`.

5.2.3. Delays and Quicks

To realistically model information propagation without dragging down computational performance, specific delays are introduced.

- **Propagation Delay**: A continuous random variable `propagation_delay` is sampled once when a post is broadcasted. It represents the time it takes for an action to travel across the network and reach followers' timelines.
- **Interaction Delay**: Sampled via `interaction_delay`, this dictates the temporal gap between a user seeing a post and effectively reacting to it.
- **Creation Delay**: Dictates the time between deciding to make a post and the actual execution of the creation event.
- **Avoidance of Continuous Time Management**: Instead of "ticking" the simulation or actively polling for state changes, delays are mathematically added to the global clock t_c exactly when the event is formulated. The resultant absolute timestamps are pushed straight into the global Priority Queue, side-stepping the need to manually track "time left until execution."

5.3. Data Structures

Essentially, as the axiom of stability remains unchanged, the graph topology structures remain static.

User Timeline (Max Heap) To satisfy the reverse-chronological consumption constraint, the timeline associated to every user is modeled as a Max Heap. This ensures that the element with the largest timestamp is always returned first. This structure must be carefully managed to prevent unbound memory growth, which is why timelines are explicitly cleared when a user logs off.

5.4. Pseudocode

The simulation relies on four main routines to govern the discrete event generation. `PropagatePost` handles the spreading of an item to followers. `StageOne` generates the initial state (the warm-up phase) to fill timelines before active simulation. `InitSessions` assigns the initial online/offline states based on a random proportion. Finally, `Simulation` executes the core loop.

```
procedure PropagatePost(u, i, t_c)
    for v ∈ N_out(u)
        if (v, i) ∉ S then
            push(T(v), gen_arrival(i, t_c))
        end
    end
end
```

```
procedure StageOne
    for u ∈ U
        push(Q, gen_create(u, t_c))
        S ← S ∪ {(u, post_count)}
    end
    while t_c ≤ t_warmup and Q ≠ ∅
        event ← pop(Q)
        t_c ← event.time
        u ← event.user
        if event.type == create then
            i ← event.post_id
            PropagatePost(u, i, t_c)
            push(Q, gen_create(u, t_c))
        end
    end
end
```

```
procedure InitSession
```

```
    for u ∈ 𝒰
        r ← Uniform(0, 1)
        if r < offline_ratio then
            online(u) ← false
            push(Q, gen_session_start(u, t_c))
        else
            online(u) ← true
            push(Q, gen_session_end(u, t_c))
        end
    end
end
```

```
procedure Simulation
    t_c, Q, 𝒮 ← 0.0, ∅, ∅
    StageOne()
    InitSessions()
    for u ∈ 𝒰 where online(u) == true
        push(Q, gen_action(u, t_c))
    end
    while t_c ≤ t_h and Q ≠ ∅
        event ← pop(Q)
        t_c, u ← event.time, event.user
        isEventStale = event.session_gen ≠ u.session_gen
        if isEventStale then continue end
        if event.type == create then
            if online(u) == true then
                i ← event.post_id
                𝒮 ← 𝒮 ∪ {(u, i)}
                PropagatePost(u, i, t_c)
            end
            push(Q, gen_create(u, t_c))
        else if event.type == session.start then
            online(u) ← true
            session_gen(u) ← session_gen(u) + 1
            push(Q, gen_session_end(u, t_c))
            push(Q, gen_action(u, t_c))
            push(Q, gen_create(u, t_c))
        else if event.type == session.end then
            online(u) ← false
            push(Q, gen_session_start(u, t_c))
```

```
        clear(𝒯(u))
    else if event.type == action and online(u) == true then
        post ← peek(𝒯(u))
        if post ≠ ∅ and post.time ≤ t_c then
            i ← post.id
            pop(𝒯(u))
            if (u, i) ∈ 𝒮 then
                push(Q, gen_action(u, t_c))
                continue
            end
            𝒮 ← 𝒮 ∪ {(u, i)}
            if event.action == repost then
                PropagatePost(u, i, t_c)
            end
            push(Q, gen_action(u, t_c))
        else
            online(u) ← false
            session_gen(u) ← session_gen(u) + 1
            push(Q, gen_session_start(u, t_c))
        end
    end
  end
end
```

5.5. Implementation

This section addresses steps taken in the design to ensure optimal performance and scalability. That is, mention that scalability is important and must be ensured at all costs. Connect with the methodology sections.

5.5.1. Data Oriented Design Basics

This implementation strictly adheres to Data-Oriented Design (DOD) principles to maximize throughput and harness modern CPU architecture.

- **Structure of Arrays (SoA)**: Instead of declaring an Array of Structs (AoS) where user properties are bundled together, Zig's `MultiArrayList` separates fields into disjoint arrays. This means boolean flags like `.is_online` are packed contiguously in memory independently of `ids`.
- **Cache Line Locality**: When the simulation iterates over millions of users to check their online status, the CPU fetches entire cache lines (typically 64 bytes). Because `.is_online` flags are tightly packed natively in the SoA, a single memory fetch loads data for 64 users simultaneously, drastically minimizing cache misses.

- **Binary Shifting and Masking**: CPU architectures evaluate bitwise operations in single clock cycles. Advanced structures in the simulation (like the segmented lists and paginated bitsets) strictly enforce power-of-two capacities, allowing the codebase to calculate boundaries using highly optimized bitwise arithmetic (`>>` and `&`) rather than costly mathematical modulo or division instructions.

5.5.2. Dynamic Dispatch of Distributions via JSON

Because the simulation configures statistical models dynamically at runtime, a system is required to parse distinct probability functions seamlessly.

- A custom `loadJson` function processes the JSON config and maps JSON distribution strings directly into `ContDist` and `DiscDist` wrapper types.
- These wrapper types utilize Zig's struct polymorphism via vtables (`vtable = &.{ .sample = sampleImpl }`) to abstract the underlying algorithms (like Ziggurat for Exponential or the Inverse Transform for Categorical).
- This dynamic dispatch guarantees that the core event loops simply call `.sample(rng)` without caring about the concrete mathematical implementation underneath.

5.5.3. Heap and n-ary Leaf

The core engine for discrete event simulation relies heavily on Priority Queues, constructed intrinsically as Heaps.

- **What is a Heap?** A heap is a specialized tree-based data structure that satisfies the heap property, meaning the highest priority element is always mathematically bound to the root node.
- **Why it's our best choice**: A heap guarantees optimal $O(\log N)$ performance for inserting new events and $O(1)$ for finding the next chronological event (or reverse-chronological post in the timeline). It ensures the simulation's central loop remains lightning-fast even with millions of pending events.
- **n-ary optimizations**: Implementing heaps as contiguous arrays ensures memory locality, while mapping tree branches mathematically prevents the necessity of pointer chasing.

5.5.4. Post Creation and Storage

Because posts can be created arbitrarily throughout the simulation duration without a hard upper limit, their tracking mechanisms must be highly scalable.

Segmented List for Posts To hold an uncapped number of posts in memory without incurring massive reallocation penalties, a `SegmentedMultiArrayList` is utilized. This behaves like a dynamically growing bookshelf: it allocates arrays in fixed capacities equal to a power of two (2^n). When one block fills up, a new one is allocated and appended. Searching for a post relies entirely on rapid bitwise operations (`i >> n` for the block, `i & (capacity - 1)` for the local index), completely avoiding array reallocation.

Additionally, this structure maintains a Structure-of-Arrays (SoA) layout. If posts eventually incorporate heavy elements like `[1536]f32` NLP embeddings, keeping data in SoA format prevents these massive arrays from polluting the CPU cache when the core simulation only needs to iterate rapidly over lightweight fields like `author_id` and `timestamp`.

Paginated Bitset for Impressions To track the impression footprint—who has seen what—we model a 2D matrix (users by posts) using a custom `PagedBitSet`. Since users are static but posts grow infinitely, this bitset allocates memory in discrete "pages." Each page covers all users vertically but limits the horizontal domain (posts) to a fixed size. As new posts are generated beyond the current bound, new pages are automatically allocated, resolving the strict memory bottleneck required by statically allocated bitsets. Note: The codebase specifically refers to this structure as `PagedBitSet` rather than "Paginated Bitset".

Post ID Assignment A scheduling quirk arises with dynamic creation: a post might be scheduled in the event queue as $(u, \text{create}, p_{\text{id}}, t)$, but because events can be dropped or skipped, the ID predicted at schedule time might not align with the actual ID when the event fires. To solve this, scheduled creates are assigned a placeholder ID of `0` in the queue, and are strictly assigned their true, globally unique ID only at the exact moment the event is actually processed.

5.5.5. Graph Representation of the Network Topology

The static follower connections forming the network topology are encoded using a Compressed Sparse Row (CSR) format, generally referred to as a Static Adjacency Array.

- Instead of giving each user their own dynamic list of followers (which incurs massive pointer overhead and heap fragmentation), all follower relations are concatenated into a single, massive, contiguous `followers: []Index` slice.
- Each `User` simply stores a `follower_start` integer and a `follower_count` integer.
- To iterate over user u's followers, the code simply slices the global array: `followers[follower_start .. follower_start + follower_count]`. This ensures maximum cache hit rates during massive post propagation storms.

5.5.6. Memory Allocation Strategies

5.6. Additional Mechanics: Quoting, Notifications and Replies

Quoting A user can decide to quote a post, establishing a new piece of content that directly references a `parent_id`.

- The newly generated quote post is immediately marked as seen by its creator and is propagated to their followers.
- Quoting is strictly a reactive action; unlike standard post generation, creating a quote does not automatically schedule subsequent post creations in the queue.
- To ensure coherent interaction flow, whenever a follower encounters a quote post in their timeline, the system automatically retrieves the original parent post and injects it at the exact same timestamp. This ensures the user has full context before responding.

Notifications Every quote actively generates a targeted notification aimed at the original author, breaking the strictly linear consumption of the timeline:

- If the target user is online when the notification arrives, the system bypasses normal propagation and immediately drops the new quote post directly into the user's active timeline to be processed next.
- If the user is offline, the system evaluates an `attend_offline_notification` probability. Upon success, the user abruptly "wakes up" out of schedule; the quote is injected into their timeline,

and a new action event is dispatched to simulate the user logging on specifically to check the interaction.
- If the offline user fails the probability check, the notification lands in a Pending Buffer. Once the user finally comes back online, they are forced to process and empty this priority buffer before returning to standard timeline consumption.

To maintain realism as a discrete event model, user stamina mechanics can be introduced: if a user logs in to a massive backlog of notifications, they might only process a subset (e.g., following a Geometric distribution) before getting bored and discarding the rest.

5.7. Trace Validation

Validating the execution of a highly stochastic and concurrent DES involves parsing the simulation's event trace.

- A validation script independently runs through the output trace (typically a CSV/JSON log of executed events) and verifies discrete logical rules.
- **Causality Checks**: The trace must guarantee that a post is inherently created before it is reposted or liked by any other entity.
- **Timeline Fidelity**: It must assert that a user never interacts with their own posts, enforcing the No-Self-Loop axiom.
- **Strict Ordering**: The timeline timestamps of consumed items must rigorously obey a monotonic reverse-chronological consumption order for every discrete session.

6. Content approach: Embeddings

As the basic engine is finished it's time to take down the most constrictive axioms. Let's recall the rellevant axioms @.

1. User Homogeneity: Every user $u \in \mathcal{U}$ is indistinguishable in behavior and shares the exact same decision policy π.
$$\forall u_i, u_j \in \mathcal{U} : \pi_{u_i} = \pi_{u_j} = \pi$$
2. Post homogeneity: All post are the same content wise, as there is no post content.
$$i = j \forall i, j \in I$$
3. Action Independence: A user's choice to interact with a post i at time t depends strictly on their static policy π, independent of their historical impression history $\mathcal{H}_u^{\text{act}}$.
$$\mathbb{P}\Big(e = (u, i, a, t) \mid \mathcal{H}_{u(t)}^{\text{act}}\Big) = \pi(a)$$

This section focuses on the decided approach on how to take down this axioms.

6.1. Main Mechanics and match with DES

DES main strength are the scalability of the simulations, which is able to process milions of elements in very few minutes. That limits our options of using posts as something that needs to not to be a heavy computational task in the middle of the simulations (*e.g.*, we cannot wait for any GPU computations on the fly to generate text, discarding LLM's use both in generation and understanding) so we have to settle for the next best thing: an embedding.

An embedding (definition of embedding)

Given a user $u \in U$ we can consider its historic ($\mathcal{H}(u)$) as a good way to define who the user is. Let's consider the posts created by the user $\mathcal{P}(u, 0)$, before the simulation starts. Considering the embedding f, we can define the user state as
$$S(u) = \Gamma(\{f(i) \mid i \in \mathcal{P}(u, 0)\})$$
where Γ is an aggregation function. Of course, it's trivial that $S(u)$ is an embedding in itself.

Once the state of the user is done its just a vector in a space. This means that we have a representation of user opinions and characteristics through a post in a single vector. With that, we can sample from this vector to generate posts with similar characteristics the user would have done, removing the way to use actual words, but the embedding being representative of the user opinions, see Section 6.2.

Now, every user can generate semantically relevant posts according to their inner state, as the embedding combination $S(u)$ allow us to represent a user as a easy-to-work-with numerical vector.

This allows to address each axiom one by one

1. User Homogeneity: we just need a series of posts to create our user. Semantic analysis of the embedding might allow for synthetic data to not need empirical data.
2. Post homogeneity: as every user samples posts now, there will be no identical posts.
3. Action Independence: see Section 6.3

6.2. Sampling Mechanisms

Method 1 - Dirichlet Weighting

The user's state $S(u)$ is defined not as a single statistical point (and with the use of the Γ function above), but as a bounded collection of the N most recently liked posts:

$$S(u) = \{i_1, i_2, ..., i_N\} \quad \text{where } i_j \in \mathcal{P}(u)$$

To generate a new post i_{N+1}, we sample a set of random weights w_j (approximating a Dirichlet distribution) such that their sum equals 1. The new post is the convex combination of the history:

$$\sum_{i=1}^{N} w_i = 1$$

$$i_{N+1} = \sum_{j=1}^{N} (w_j \cdot i_j)$$

By changing with posts are in $S(u)$ we can also change user state, and according to the weight of that post it will have more or less impact.

Method 2 - Gaussian Sampling

The user's state consists of a continuously updated mean μ and variance σ^2 calculated independently for all dimensions. This can be obtained with the application of a Γ function combination of the embedded posts.

When a user interacts with a post i, we update their profile using a learning rate α. The variance calculation is adapted from Welford's online algorithm to maintain numerical stability:

$$\mu_{j+1} = (1-\alpha)\mu_j + \alpha i$$

$$\sigma_{j+1}^2 = (1-\alpha)\sigma_j^2 + \alpha\big((P-\mu_j) \odot (P-\mu_{j+1})\big)$$

Note: $\odot$ represents the Hadamard product (element-wise multiplication).

To create a new post, we draw standard normal noise $\varepsilon \sim \mathcal{N}(0,1)$ and scale it by the dynamic variance:

$$i = \mu + (\sigma \odot \varepsilon)$$

6.3. Action Independence

This section adapts based on the sampling approach defined in Section 6.2.

Action independence asserts that a user's current action does not depend on their chronological sequence of past actions, but is instead sampled independently from a behavioral policy:

$$P\Big(e = (u, i, a, t) \mid \mathcal{H}_{u(t)}^{\text{act}}\Big) = \pi_{u(a)}$$

However, rather than using static probabilities for every interaction, we want this policy to dynamically react to the actual content the user is viewing. For example: if a post perfectly aligns with a user's interests, the probability of doing "Nothing" should drop drastically, while "Like" and "Repost" should rise. Conversely, if the post is completely irrelevant, "Nothing" should become the overwhelmingly likely outcome. That is, we want the thresholds of the categorical distribution to change when according to the semantics of the post

We define this alignment by calculating the cosine similarity c between the user's current state $S(u)$ and the post's embedding $f(i)$:

$$c = S_C(S(u), f(i)) = \frac{\boldsymbol{S(u)} \cdot \boldsymbol{f(i)}}{\|\boldsymbol{S(u)}\| \, \|\boldsymbol{f(i)}\|}$$

2. Defining Action Parameters (Bias and Sensitivity)

To map this similarity score c into a concrete decision, we calculate a raw mathematical score (a "logit", denoted as z_a) for every possible action $a \in \mathcal{A}$ (e.g., Nothing, Like, Repost). This score is built using two intuitive parameters:

- **Base Bias ($\boldsymbol{\beta_a}$):** This represents the default tendency of the user to take this action, regardless of the content. Because users naturally scroll past the vast majority of posts, the "Nothing" action is assigned a very high base bias, while "Repost" is assigned a low base bias.
- **Sensitivity ($\boldsymbol{\theta_a}$):** This acts as a multiplier, determining how strongly the action reacts to the content similarity c. "Like" and "Repost" will have high positive sensitivities (meaning a high c rapidly drives up their score), whereas "Nothing" has a negative sensitivity (a high c actively reduces its likelihood).

$$z_a = \theta_a \cdot c + \beta_a$$

3. The Activation Function (Softmax)

Finally, these raw scores (z_a) are unbounded numbers and do not represent true probabilities. To convert them into a valid probability distribution—where all options are between 0% and 100% and sum exactly to 1.0—we pass them through an **activation function** called Softmax:

$$\pi_u(a \mid c) = \frac{\exp(z_a)}{\sum_{k \in \mathcal{A}} \exp(z_k)}$$

This formulation avoids the rigidity of deterministic thresholds. It ensures that a high similarity c exponentially boosts the likelihood of engagement, while preserving the stochastic, random noise inherent to authentic human browsing behavior.